Настоящее учебно-методическое пособие предназначено для студентов высших учебных заведений, изучающих дисциплину «Базы данных». Пособие сохраняет фундаментальные темы курса — проектирование, нормализацию, язык SQL, обеспечение целостности — и одновременно приближает практикум к современным требованиям отрасли. В качестве сквозного учебного кейса используется разработка базы данных маркетплейса (аналог Ozon, Wildberries, Яндекс Маркет), что позволяет изучать технологии на задачах, близких к реальной разработке информационных систем. В качестве СУБД применяется PostgreSQL.

Пособие состоит из общих теоретических разделов и семи практических работ: от анализа предметной области и проектирования схемы до оконной аналитики, серверной логики, работы с JSONB и оптимизации запросов. Каждая лабораторная работа содержит развёрнутую теоретическую часть, достаточную для выполнения задания и ответа на контрольные вопросы, техническое задание на основе сквозного кейса и контрольные вопросы для самопроверки и подготовки к защите.

Ключевые слова: базы данных, PostgreSQL, SQL, ER-моделирование, нормализация, JOIN, оконные функции, транзакции, ACID, PL/pgSQL, триггеры, индексы, EXPLAIN, JSONB, маркетплейс.

Содержание

Содержание 2
Введение 5
Сквозной кейс: предметная область маркетплейса 7
Инструменты разработки 9
Система контроля версий 11
Требования к выполнению лабораторных работ 12
Лабораторная работа №1 13
Проектирование и создание структуры базы данных 13
Теоретическая часть 13
Реляционная модель данных 13
Этапы проектирования и ER-модель 13
Ключи 14
Связи и ссылочная целостность 14
Ограничения целостности 15
Нормализация 15
Иерархические данные и реализация схемы 16
Техническое задание 17
Контрольные вопросы 18
Лабораторная работа №2 19
Базовые SQL-запросы и фильтрация данных 19
Теоретическая часть 19
Структура команды SELECT 19
Фильтрация строк: WHERE и предикаты 19
Значение NULL и трёхзначная логика 20
Сортировка, ограничение, уникальность 20
Наполнение таблиц данными 20
Техническое задание 21
Контрольные вопросы 21
Лабораторная работа №3 23
Многотабличные запросы и аналитические представления 23
Теоретическая часть 23
Соединение таблиц (JOIN) 23

Агрегатные функции и группировка 23
Подзапросы 24
Общие табличные выражения (CTE) 24
Представления (VIEW) 25
Техническое задание 25
Контрольные вопросы 26
Лабораторная работа №4 27
Оконные функции и аналитика данных 27
Теоретическая часть 27
Понятие оконной функции 27
Синтаксис: конструкция OVER 27
Ранжирующие функции 27
Агрегатные функции в роли оконных и функции смещения 28
Топ-N в группе и сравнение с GROUP BY 28
Техническое задание 28
Контрольные вопросы 29
Лабораторная работа №5 30
Транзакции, целостность данных и серверная логика 30
Теоретическая часть 30
Транзакции и свойства ACID 30
Функции и язык PL/pgSQL 31
Триггеры 31
Логика в базе данных или в приложении 32
Техническое задание 32
Контрольные вопросы 33
Лабораторная работа №6 35
Современные возможности PostgreSQL и оптимизация запросов 35
Теоретическая часть 35
Тип данных JSONB 35
Как СУБД исполняет запрос 35
Индексы 36
Анализ плана: EXPLAIN и EXPLAIN ANALYZE 36
Цена индексов и GIN 36

Техническое задание 37
Контрольные вопросы 38
Лабораторная работа №7 39
Итоговый проект 39
Общее описание 39
Состав работы 39
Критерии оценивания 39

Введение

База данных — это организованная совокупность структурированных данных, хранимых и управляемых так, чтобы их можно было эффективно сохранять, изменять и извлекать. Системой управления базами данных (СУБД) называют программный комплекс, который обеспечивает создание базы данных, целостность хранимых данных, многопользовательский доступ и выполнение запросов. Умение проектировать базы данных и работать с ними — одна из базовых компетенций разработчика информационных систем: практически любое приложение, от мобильного клиента до крупного маркетплейса, опирается на надёжное хранилище данных.

В основе курса лежит реляционная модель данных и язык SQL. Реляционная модель описывает данные в виде набора связанных таблиц с чётко заданной структурой и правилами целостности, а SQL предоставляет декларативный способ описывать, какие данные нужно получить или изменить, не указывая, как именно СУБД должна это сделать. Такой подход остаётся отраслевым стандартом и лежит в основе большинства промышленных систем.

В качестве СУБД используется PostgreSQL — свободная объектно-реляционная система, сочетающая строгое следование стандарту SQL с богатым набором современных возможностей: оконными функциями, общими табличными выражениями (CTE), многоверсионным управлением конкурентным доступом (MVCC), процедурным языком PL/pgSQL, типом JSONB и развитой системой индексов. Этот набор позволяет в рамках одного инструмента пройти путь от простых выборок до серверной аналитики и оптимизации производительности.

Сквозным учебным кейсом служит база данных маркетплейса — площадки, на которой продавцы размещают товары, а покупатели оформляют заказы. Все семь работ развивают одну и ту же базу данных: спроектированная в первой работе схема последовательно наполняется данными, обрастает аналитическими запросами, серверной логикой и оптимизируется по производительности.

По результатам освоения курса студент должен уметь:

— проектировать структуры баз данных на основе анализа предметной области;

—создавать и настраивать реляционные базы данных средствами PostgreSQL;

—разрабатывать SQL-запросы различной сложности, включая многотабличные и аналитические выборки;

—использовать современные возможности PostgreSQL: оконные функции, CTE, JSONB;

—обеспечивать целостность данных декларативными и процедурными средствами;

—реализовывать бизнес-логику на стороне СУБД — функции и триггеры на языке PL/pgSQL;

—оптимизировать производительность запросов с помощью индексов и анализа планов выполнения;

—разрабатывать и защищать законченный проект базы данных для реального цифрового продукта.

Сквозной кейс: предметная область маркетплейса

Маркетплейс — электронная площадка, объединяющая множество независимых продавцов и покупателей. Продавцы размещают товары в каталоге, покупатели находят их, добавляют в корзину и оформляют заказы, после чего товары оплачиваются и доставляются. На протяжении всех работ разрабатывается база данных, обслуживающая такую площадку.

В предметной области выделяются следующие основные сущности, которые станут таблицами проектируемой базы данных:

—Пользователи (users) — учётные записи площадки; пользователь может быть покупателем, а также владельцем магазина-продавца.

—Продавцы (sellers) — магазины, размещающие товары; связаны с пользователем-владельцем.

—Категории (categories) — иерархический классификатор товаров (категория может иметь родительскую категорию).

—Товары (products) — карточки товаров, принадлежащие продавцу и отнесённые к категории.

—Складские остатки (stock) — количество доступного к продаже товара.

—Корзины (carts) и позиции корзины (cart_items) — отложенные к покупке товары.

—Заказы (orders) и позиции заказа (order_items) — оформленные покупки; связь товаров и заказов «многие-ко-многим».

—Платежи (payments) — сведения об оплате заказов.

—Отзывы (reviews) — оценки и комментарии покупателей к товарам.

—Адреса (addresses) и доставки (deliveries) — куда и как доставляется заказ.

Эта предметная область сознательно богата связями всех видов: «один-ко-многим» (продавец — товары), «многие-ко-многим» (заказы — товары через позиции), иерархия (категории), что делает её удобной основой для изучения проектирования, многотабличных запросов и аналитики.

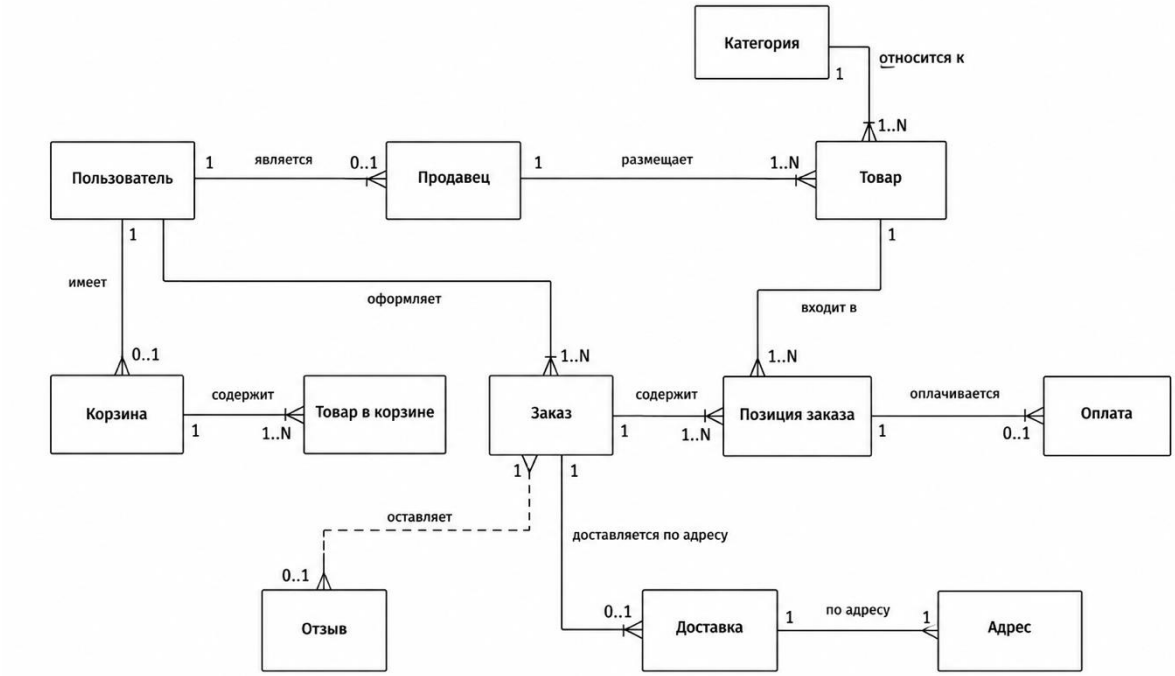


Рисунок 1 — Концептуальная ER-модель предметной области маркетплейса

Инструменты разработки

PostgreSQL — свободная объектно-реляционная СУБД с открытым исходным кодом, развиваемая сообществом более тридцати лет. Она доступна на всех основных операционных системах, строго следует стандарту SQL и поддерживает расширенные возможности, изучаемые в курсе. Для выполнения работ достаточно локально установить сервер PostgreSQL и один из клиентов для подключения и выполнения запросов.

psql — официальный клиент командной строки, входящий в поставку PostgreSQL. Он позволяет подключаться к базе, выполнять SQL-команды и служебные мета-команды: \l — список баз, \dt — список таблиц, \d имя_таблицы — структура таблицы, \timing — измерение времени выполнения. psql незаменим для скриптов и автоматизации.

Для повседневной работы удобны графические клиенты. pgAdmin — официальное веб-приложение для администрирования PostgreSQL с деревом объектов, редактором запросов и средствами просмотра планов выполнения. DBeaver — универсальный клиент с автодополнением и визуализацией данных. Выбор клиента не влияет на содержание работ: все задания выполняются стандартным SQL.

Типовой порядок начала работы: установить сервер PostgreSQL; создать учебную базу командой CREATE DATABASE marketplace; подключиться к ней; выполнять SQL-скрипты работ. Рекомендуется оформлять все команды в виде сохраняемых скриптов (.sql), чтобы базу можно было воссоздать с нуля и воспроизвести результаты.

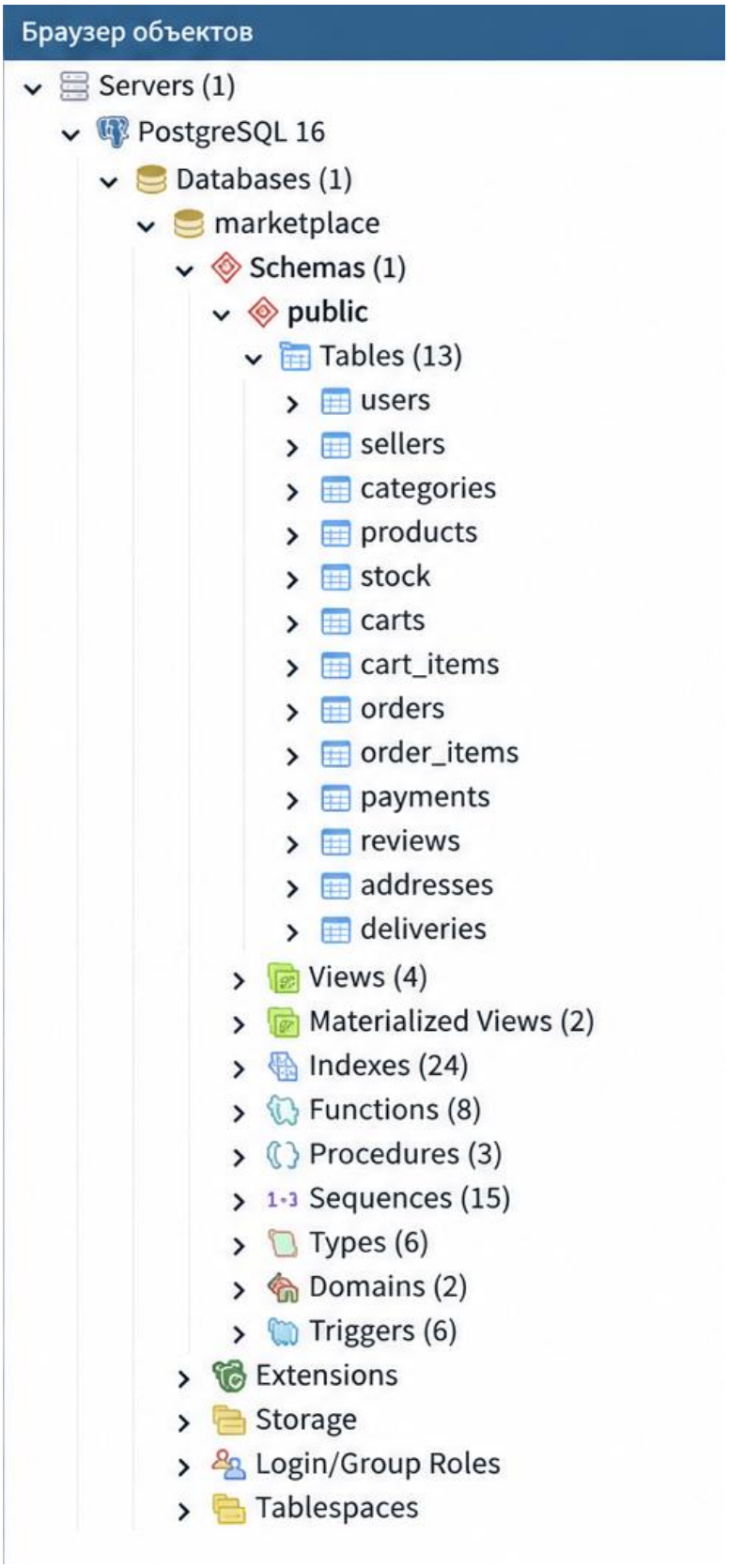


Рисунок 2 — Дерево объектов базы данных в графическом клиенте

Система контроля версий

Git — распределённая система контроля версий, предназначенная для ведения истории изменений в файлах проекта. Она фиксирует состояние проекта во времени и позволяет отслеживать, кто, когда и какие изменения внёс. При выполнении работ Git используется для хранения SQL-скриптов, отчётов и итогового проекта, а также для сдачи работ преподавателю.

В базовом цикле применяются три команды. git add добавляет изменённые файлы в индекс (помечает, что попадёт в фиксацию). git commit сохраняет подготовленные изменения в локальной истории в виде коммита с поясняющим сообщением. git push отправляет локальные коммиты в удалённый репозиторий.

Ветвление позволяет вести разработку параллельными линиями. Под каждую работу удобно создавать отдельную ветку (например, lab_1), а по завершении объединять её с основной через pull request на платформе хостинга (GitHub, GitLab). Pull request фиксирует предлагаемые изменения, позволяет провести их проверку и затем выполнить слияние.

Требования к выполнению лабораторных работ

1. Все работы выполняются в СУБД PostgreSQL (рекомендуемая версия — 15 или новее).
2. Результат каждой работы оформляется в виде SQL-скрипта (скриптов), который можно выполнить повторно и воспроизвести результат.
3. Все объекты базы данных именуются осмысленно, на латинице, в едином стиле (рекомендуется snake_case).
4. К работе прилагается краткий отчёт: постановка задачи, принятые решения и их обоснование, тексты ключевых запросов и результаты их выполнения.
5. Скрипты и отчёт размещаются в репозитории системы контроля версий; каждая работа — в отдельной ветке или папке.
6. Сквозная база данных развивается от работы к работе: схема, созданная в первой работе, используется во всех последующих.
7. Защита работы предполагает не только демонстрацию результата, но и объяснение принятых решений и ответы на контрольные вопросы.

Лабораторная работа №1

Проектирование и создание структуры базы данных

Теоретическая часть

Реляционная модель данных

Реляционная модель, предложенная Э. Коддом, представляет данные в виде отношений. Отношение на практике — это таблица, состоящая из строк и столбцов. Строку называют также кортежем или записью: она описывает один объект предметной области, например один товар. Столбец называют атрибутом или полем: он описывает одно свойство объекта и связан с доменом — множеством допустимых значений, которое в СУБД задаётся типом данных и ограничениями. Совокупность имён столбцов и их типов образует схему отношения; она фиксируется при создании таблицы и определяет, какие данные допустимо в ней хранить. Важные свойства отношения: строки не упорядочены (порядок задаётся только запросом), а каждое значение атомарно — в одной ячейке хранится одно неделимое значение.

Этапы проектирования и ER-модель

Проектирование базы данных проходит через три уровня моделей. Концептуальная модель описывает сущности и связи между ними без привязки к конкретной СУБД и обычно изображается ER-диаграммой (Entity — Relationship, «сущность — связь»). Логическая модель уточняет атрибуты каждой сущности, первичные и внешние ключи, типы связей. Физическая модель — это конкретные таблицы PostgreSQL с типами данных и ограничениями, готовые к созданию командами DDL. Такое разделение позволяет сначала договориться о составе данных, не отвлекаясь на технические детали, и лишь затем перейти к реализации.

На ER-диаграмме сущность — это объект, о котором хранят данные (товар, заказ, пользователь); атрибут — его свойство (наименование, цена); связь отражает отношение между сущностями. Для связи указывают кардинальность — сколько объектов одной сущности соответствует объекту другой. Различают три вида связей. «Один-к-одному» (1:1) встречается редко и обычно выносит редкие или чувствительные атрибуты в отдельную таблицу. «Один-ко-многим» (1:N) — самый частый вид: одному продавцу принадлежит много товаров. «Многие-ко-многим» (M:N) означает, что объектам с обеих сторон соответствует множество объектов

другой стороны: один заказ содержит много товаров, а один товар входит во многие заказы.

Ключи

Каждая строка таблицы должна однозначно идентифицироваться. За это отвечает первичный ключ (PRIMARY KEY) — столбец или набор столбцов, значения которых уникальны и не равны NULL. Требования к первичному ключу: уникальность (не бывает двух строк с одинаковым ключом), обязательность (значение всегда задано), стабильность (значение не должно меняться со временем). Если уникальных столбцов несколько, их называют потенциальными (candidate) ключами; один из них выбирают первичным, остальные остаются альтернативными и поддерживаются ограничением UNIQUE.

В качестве первичного ключа используют суррогатный или естественный ключ. Суррогатный ключ — искусственный идентификатор, не несущий смысла предметной области (целочисленный автоинкремент или UUID). Его достоинства: стабильность (он никогда не меняется), компактность, независимость от изменений бизнес-правил. Недостаток — он не нагляден и требует соединения с другими таблицами, чтобы понять смысл строки. Естественный ключ образован из реальных атрибутов (например, артикул товара или email пользователя). Он нагляден и иногда уже уникален по смыслу, но менее устойчив: реальные данные имеют свойство меняться и иногда оказываются не такими уникальными, как предполагалось. В учебной схеме рекомендуется суррогатный ключ, объявляемый как GENERATED ALWAYS AS IDENTITY.

Связи и ссылочная целостность

Связи между таблицами реализуются внешними ключами (FOREIGN KEY). Внешний ключ — столбец, значения которого ссылаются на первичный ключ другой (или той же) таблицы. Например, в таблице товаров столбец seller_id ссылается на первичный ключ таблицы продавцов. Внешний ключ обеспечивает ссылочную целостность: СУБД не позволит вставить товар с несуществующим продавцом и контролирует судьбу зависимых строк при изменении или удалении родительской.

Поведение при удалении или изменении родительской строки задаётся правилами ON DELETE и ON UPDATE. ON DELETE RESTRICT (и близкое NO ACTION) запрещает удаление родителя, пока на него ссылаются дочерние строки. ON DELETE CASCADE удаляет дочерние строки

автоматически вслед за родителем. ON DELETE SET NULL обнуляет внешний ключ в дочерних строках, ON DELETE SET DEFAULT — подставляет значение по умолчанию. Выбор правила — проектное решение: удаление товара не должно молча удалять позиции уже оформленных заказов (здесь уместен RESTRICT), а удаление заказа логично каскадно удаляет его позиции (CASCADE). Связь «многие-ко-многим» реализуется ассоциативной таблицей с двумя внешними ключами; её первичным ключом обычно служит пара этих ключей.

Ограничения целостности

Целостность данных бывает доменной (значение принадлежит допустимому множеству), сущностной (каждая строка уникально идентифицируется первичным ключом) и ссылочной (внешние ключи указывают на существующие строки). PostgreSQL поддерживает декларативные ограничения, которые задаются один раз в схеме и автоматически проверяются при любых операциях:

—PRIMARY KEY — первичный ключ (уникальность и NOT NULL одновременно);

—FOREIGN KEY — внешний ключ (ссылочная целостность);

—NOT NULL — запрет отсутствия значения;

—UNIQUE — уникальность значений столбца (например, email пользователя);

—CHECK — произвольное логическое условие на значение (например, price > 0);

—DEFAULT — значение по умолчанию при отсутствии явного.

Нормализация

Нормализация — процесс приведения схемы к виду, при котором сокращается избыточность и устраняются аномалии: вставки (нельзя добавить данные об объекте, пока нет связанного с ним), обновления (одно и то же значение приходится менять во многих строках, рискуя рассогласовать данные) и удаления (удаление одной строки теряет полезные сведения о другом объекте). Источник аномалий — смешение в одной таблице независимых сущностей и наличие нежелательных функциональных зависимостей между атрибутами.

Первая нормальная форма (1НФ) требует атомарности значений: никаких списков и повторяющихся групп в одной ячейке. Вторая

нормальная форма (2НФ) требует, чтобы каждый неключевой атрибут зависел от всего составного первичного ключа, а не от его части (актуально только при составном ключе). Третья нормальная форма (3НФ) дополнительно запрещает транзитивные зависимости — когда неключевой атрибут зависит от другого неключевого. Нормальная форма Бойса — Кодда (БКНФ) усиливает 3НФ, требуя, чтобы любая функциональная зависимость исходила из потенциального ключа. На практике корректная схема в 3НФ устраняет большинство аномалий и считается базовым целевым уровнем. Иногда ради производительности применяют осознанную денормализацию — намеренно вводят избыточность (например, хранят заранее вычисленный рейтинг товара), понимая, что взамен придётся поддерживать согласованность дублированных данных.

Иерархические данные и реализация схемы

Иерархию (дерево категорий) моделируют ссылкой таблицы на саму себя: столбец parent_id является внешним ключом на первичный ключ той же таблицы. Корневые категории имеют parent_id, равный NULL. Такой способ называют списком смежности; для обхода всего поддерева применяют рекурсивные запросы (рассматриваются в работе №3). Пример определения таблицы категорий с самоссылкой, таблицы товаров и ассоциативной таблицы позиций заказа:

```
CREATE TABLE categories (
    id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_id BIGINT REFERENCES categories(id) ON DELETE RESTRICT,
    name      TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    seller_id   BIGINT NOT NULL REFERENCES sellers(id) ON DELETE RESTRICT,
    category_id BIGINT NOT NULL REFERENCES categories(id),
    title       TEXT NOT NULL,
    price       NUMERIC(12,2) NOT NULL CHECK (price > 0),
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity   INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price > 0),
    PRIMARY KEY (order_id, product_id)
);
```

PostgreSQL предоставляет богатый набор типов данных: целочисленные (INTEGER, BIGINT), числа с фиксированной точностью (NUMERIC) для денежных величин, строковые (TEXT, VARCHAR), логический (BOOLEAN), дата и время (DATE, TIMESTAMPTZ — момент времени с часовым поясом), а также UUID, массивы и JSONB. Цену хранят в NUMERIC, а не в типах с плавающей точкой, чтобы избежать ошибок округления денежных сумм.

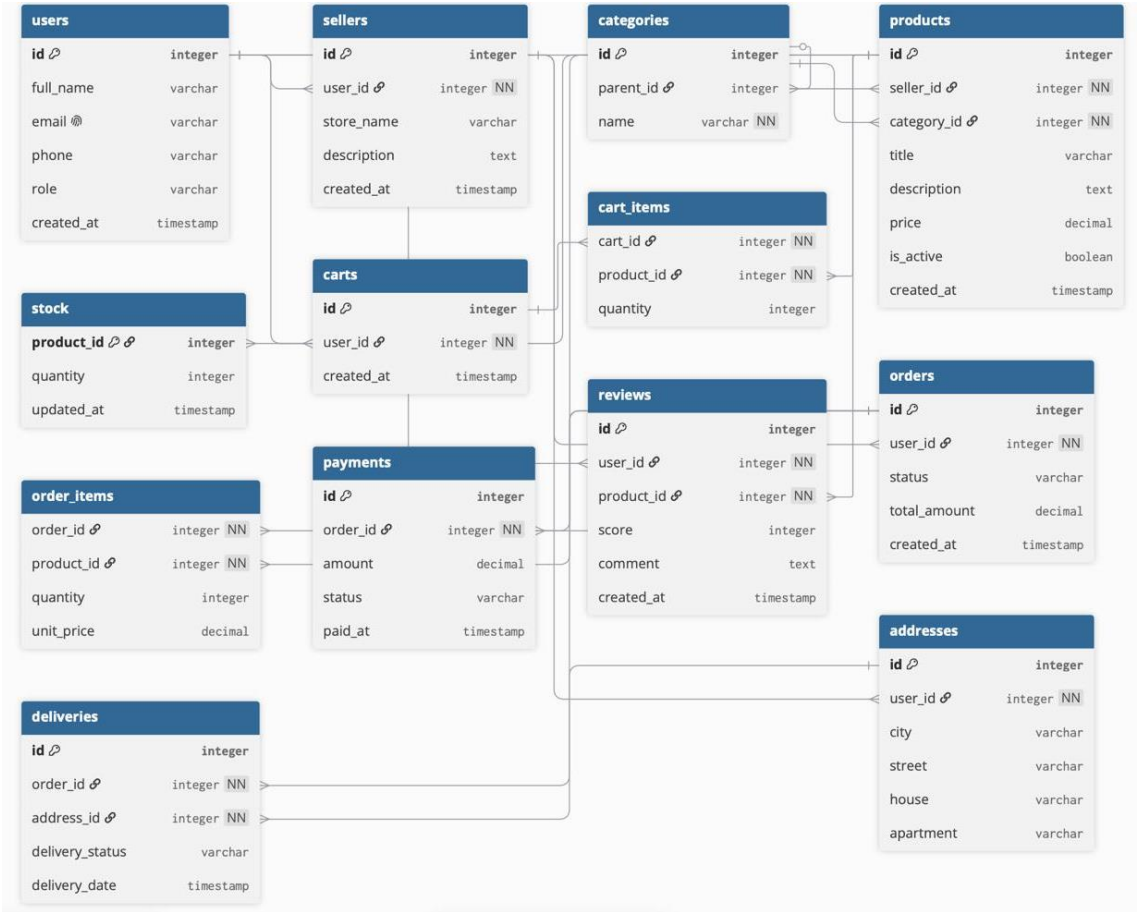


Рисунок 3 — Логическая схема базы данных (диаграмма «сущность — связь»)

Техническое задание

Спроектируйте и реализуйте структуру базы данных маркетплейса согласно описанию предметной области.

1. Проведите анализ предметной области и сформулируйте перечень бизнес-правил (например: у товара ровно один продавец; заказ принадлежит одному покупателю и содержит хотя бы одну позицию; отзыв оставляется к конкретному товару конкретным пользователем).
2. Постройте ER-диаграмму, включающую не менее десяти сущностей. Схема должна содержать хотя бы одну связь «многие-ко-многим»

через ассоциативную таблицу и хотя бы одну иерархическую связь (самоссылку).

3. Приведите схему к третьей нормальной форме. На примере одной таблицы покажите аномалию до нормализации и поясните, как она устранена.
4. Определите для каждой таблицы первичный ключ; обоснуйте выбор суррогатных или естественных ключей.
5. Реализуйте схему в PostgreSQL средствами DDL: создайте таблицы с корректными типами, первичными и внешними ключами, ограничениями NOT NULL, UNIQUE, CHECK, DEFAULT и правилами ON DELETE / ON UPDATE.
6. Составьте словарь данных: для каждой таблицы перечислите столбцы, их типы, ограничения и назначение.
7. Оформите все команды единым SQL-скриптом, повторное выполнение которого воссоздаёт схему с нуля.

Контрольные вопросы

1. Что такое реляционная модель данных и что в ней называют отношением, кортежем и атрибутом?
2. Чем отличаются концептуальная, логическая и физическая модели данных?
3. Что такое первичный ключ и какие требования к нему предъявляются?
4. В чём различие между суррогатным и естественным ключом? Каковы их достоинства и недостатки?
5. Что такое внешний ключ и как он обеспечивает ссылочную целостность?
6. Как в реляционной модели реализуется связь «многие-ко-многим»?
7. Как смоделировать иерархические данные (дерево категорий) одной таблицей?
8. Какие ограничения целостности поддерживает PostgreSQL и что задаёт каждое из них?
9. Что такое нормализация и какие аномалии она устраняет? Сформулируйте требования 1НФ, 2НФ и 3НФ.
10. Для чего нужны правила ON DELETE RESTRICT и ON DELETE CASCADE и чем они различаются?

Лабораторная работа №2

Базовые SQL-запросы и фильтрация данных

Теоретическая часть

Структура команды SELECT

SELECT — основная команда выборки данных. В полном виде она состоит из секций, каждая из которых отвечает за свой этап обработки: SELECT (какие столбцы и выражения вернуть), FROM (источник данных), WHERE (фильтрация строк), GROUP BY и HAVING (группировка и фильтрация групп), ORDER BY (сортировка), LIMIT/OFFSET (ограничение количества). Важно различать порядок записи и логический порядок выполнения: сначала отрабатывает FROM, затем WHERE, далее группировка, потом вычисляется список SELECT, после — ORDER BY и в конце LIMIT. Поэтому, например, псевдоним столбца, заданный в SELECT, доступен в ORDER BY, но не в WHERE.

В списке выборки перечисляют конкретные столбцы, выражения над ними и литералы; столбцам и выражениям можно давать псевдонимы через ключевое слово AS. Символ * означает «все столбцы», но в промышленном коде его избегают: явное перечисление полей устойчивее к изменениям схемы, понятнее читателю и эффективнее, так как не передаёт лишние данные.

```
SELECT title AS name, price, price * 0.9 AS discounted
FROM products;
```

Фильтрация строк: WHERE и предикаты

Секция WHERE задаёт логическое условие; в результат попадают строки, для которых оно истинно. В условиях используются операторы сравнения (=, <>, <, >, <=, >=) и специальные предикаты. BETWEEN a AND b задаёт замкнутый диапазон и эквивалентен записи (x >= a AND x <= b), но короче и нагляднее. IN (...) проверяет принадлежность значения списку и заменяет цепочку условий через OR. LIKE сопоставляет строку с шаблоном: символ % заменяет любую последовательность символов (в том числе пустую), а _ — ровно один символ; оператор ILIKE делает то же без учёта регистра. Сложные условия комбинируются операторами AND, OR и NOT, а порядок их вычисления уточняется скобками (AND связывает сильнее, чем OR).

```
SELECT title, price
FROM products
```

```
WHERE category_id IN (1, 2, 5)
  AND is_active = TRUE
  AND price BETWEEN 1000 AND 5000
  AND title ILIKE '%телефон%';
```

Значение NULL и трёхзначная логика

NULL обозначает отсутствие значения (неизвестно или неприменимо) и не равно ничему, даже другому NULL. Поэтому SQL использует трёхзначную логику: результатом сравнения может быть истина, ложь или «неизвестно» (UNKNOWN). Сравнение с NULL обычными операторами всегда даёт UNKNOWN, и такие строки в результат WHERE не попадают. Для проверки отсутствия значения применяют предикаты IS NULL и IS NOT NULL. Функция COALESCE(x, y) возвращает первое не-NULL значение из списка и удобна для подстановки значений по умолчанию.

```
SELECT title, COALESCE(description, 'нет описания') AS descr
FROM products
WHERE description IS NULL;
```

Сортировка, ограничение, уникальность

Секция ORDER BY сортирует результат по одному или нескольким столбцам или выражениям; направление задаётся ASC (по возрастанию, по умолчанию) и DESC (по убыванию). Для каждого ключа сортировки можно отдельно указать положение пустых значений: NULLS FIRST или NULLS LAST. Сортировать можно по псевдониму столбца или по его порядковому номеру в списке выборки.

Секции LIMIT и OFFSET ограничивают число возвращаемых строк и задают смещение от начала; вместе они реализуют постраничный вывод (например, LIMIT 20 OFFSET 20 — вторая страница по 20 записей). Следует помнить, что большие значения OFFSET неэффективны, так как СУБД всё равно вычисляет и отбрасывает пропускаемые строки. Ключевое слово DISTINCT в списке выборки убирает полностью совпадающие строки, оставляя только различные; конструкция DISTINCT ON (столбец) оставляет по одной строке на каждое значение указанного столбца.

```
SELECT title, price
FROM products
WHERE is_active
ORDER BY price DESC, title ASC
LIMIT 20 OFFSET 20;
```

Наполнение таблиц данными

Перед выборками таблицы наполняют командой INSERT. Она добавляет одну или несколько строк, перечисленных в VALUES, либо

строки, полученные другим запросом (INSERT ... SELECT). Для генерации большого объёма тестовых данных удобна функция generate_series, возвращающая последовательность чисел, в сочетании с функцией random(). Пример массового наполнения таблицы товаров тысячей строк:

```
INSERT INTO categories (name) VALUES ('Электроника'), ('Книги'), ('Одежда');

INSERT INTO products (seller_id, category_id, title, price)
SELECT 1,
       (random() * 2 + 1)::int,
       'Товар №' || g,
       (random() * 9000 + 1000)::numeric(12,2)
FROM generate_series(1, 1000) AS g;
```

Техническое задание

Наполните спроектированную базу тестовыми данными и составьте набор запросов на выборку и фильтрацию. В крупные таблицы (товары, заказы) внесите не менее тысячи строк.

1. Наполните все таблицы согласованными данными, используя generate_series или иной генератор для массового заполнения.
2. Напишите запрос, выводящий заданные столбцы товаров без дубликатов (DISTINCT), с псевдонимами столбцов.
3. Напишите запрос с фильтрацией по нескольким условиям, объединённым AND и OR со скобками.
4. Примените в запросах предикаты BETWEEN, IN, LIKE (или ILIKE) и IS NULL — каждый хотя бы один раз.
5. Напишите запрос с сортировкой по двум столбцам в разных направлениях.
6. Реализуйте постраничный вывод товаров с помощью LIMIT и OFFSET.
7. Для каждого запроса приведите его текст, назначение и фрагмент результата.

Контрольные вопросы

1. Из каких секций состоит команда SELECT и в каком порядке они логически выполняются?
2. Почему в промышленных запросах не рекомендуется использовать SELECT *?
3. Для чего предназначена секция WHERE и как в ней комбинируются условия?

4. Чем предикаты BETWEEN и IN отличаются от записи тех же условий через операторы сравнения и OR?
5. Как работает оператор LIKE и что означают символы % и _ в шаблоне?
6. Что такое трёхзначная логика и почему нельзя проверять значение на NULL оператором «=»?
7. Что делает секция ORDER BY и как задать сортировку по убыванию и положение NULL?
8. Для чего служат LIMIT и OFFSET и как с их помощью организовать постраничный вывод?
9. Что делает ключевое слово DISTINCT?
10. Как сгенерировать большой объём тестовых данных средствами PostgreSQL?

Лабораторная работа №3

Многотабличные запросы и аналитические представления

Теоретическая часть

Соединение таблиц (JOIN)

Данные в нормализованной базе распределены по нескольким таблицам, поэтому для осмысленного результата их соединяют. Соединение (JOIN) объединяет строки двух таблиц по условию, заданному в секции ON, — обычно по равенству внешнего ключа одной таблицы и первичного ключа другой. Таблицам удобно давать короткие псевдонимы и обращаться к столбцам через них. Если столбцы соединения называются одинаково, можно использовать сокращённую форму USING (имя_столбца).

Внутреннее соединение INNER JOIN оставляет только те пары строк, для которых условие истинно; строки без соответствия отбрасываются. Левое внешнее соединение LEFT JOIN сохраняет все строки левой таблицы, подставляя NULL вместо отсутствующих соответствий справа, — оно нужно, чтобы показать, например, всех продавцов, включая тех, у кого ещё нет товаров. RIGHT JOIN симметричен, FULL JOIN сохраняет строки обеих таблиц. CROSS JOIN даёт декартово произведение (все сочетания строк). Самосоединение (SELF JOIN) соединяет таблицу с самой собой и применяется, например, для работы с иерархией категорий.

```
SELECT p.title, c.name AS category, s.name AS seller
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
LEFT JOIN sellers AS s ON s.id = p.seller_id;
```

Агрегатные функции и группировка

Агрегатные функции вычисляют одно сводное значение по множеству строк: COUNT — количество, SUM — сумму, AVG — среднее, MIN и MAX — наименьшее и наибольшее. Важно различать COUNT(*), который считает все строки, и COUNT(столбец), который не учитывает строки с NULL в этом столбце; прочие агрегатные функции также игнорируют NULL. Внутри агрегатной функции допустимо DISTINCT — например, COUNT(DISTINCT seller_id) считает число различных продавцов.

Секция GROUP BY делит строки на группы по значениям перечисленных столбцов; для каждой группы агрегатная функция возвращает одно значение. В списке выборки при группировке допустимы только столбцы из GROUP BY и агрегатные функции. Секция HAVING

фильтрует уже сгруппированные данные по результату агрегации, тогда как WHERE фильтрует исходные строки до группировки. Логически WHERE отрабатывает раньше группировки, а HAVING — после неё.

```
SELECT c.name, COUNT(*) AS products_count, AVG(p.price) AS avg_price
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
WHERE p.is_active
GROUP BY c.name
HAVING COUNT(*) > 100
ORDER BY products_count DESC;
```

Подзапросы

Подзапрос — запрос, вложенный в другой запрос. Скалярный подзапрос возвращает ровно одно значение и используется в списке выборки или в условии (например, сравнение цены товара со средней по базе). Подзапрос в предикате IN проверяет принадлежность значения множеству строк, возвращаемых подзапросом; операторы ANY и ALL сравнивают значение со всеми элементами множества. Предикат EXISTS истинен, если подзапрос вернул хотя бы одну строку, и удобен для проверки существования связанных данных; NOT EXISTS проверяет их отсутствие. Подзапрос в секции FROM (производная таблица) используется как временный источник строк.

Подзапросы делятся на некоррелированные и коррелированные. Некоррелированный вычисляется один раз и не зависит от внешнего запроса. Коррелированный ссылается на столбцы внешнего запроса и логически вычисляется для каждой его строки — он мощнее, но потенциально дороже.

```
-- товары дороже средней цены по базе (скалярный подзапрос)
SELECT title, price FROM products
WHERE price > (SELECT AVG(price) FROM products);


-- продавцы, у которых есть хотя бы один товар (EXISTS, коррелированный)
SELECT s.name FROM sellers AS s
WHERE EXISTS (SELECT 1 FROM products p WHERE p.seller_id = s.id);
```

Общие табличные выражения (CTE)

Общее табличное выражение (CTE), вводимое словом WITH, именует промежуточный результат и позволяет обращаться к нему как к таблице в основном запросе. CTE делает сложные запросы читаемыми, разбивая их на логические шаги; в одном запросе можно объявить несколько CTE. Рекурсивный CTE (WITH RECURSIVE) ссылается сам на себя и

применяется для обхода иерархий — например, чтобы получить все подкатегории заданной категории.

```
WITH seller_rev AS (
    SELECT p.seller_id, SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p JOIN order_items oi ON oi.product_id = p.id
    GROUP BY p.seller_id
)
SELECT s.name, r.revenue,
       r.revenue / SUM(r.revenue) OVER () AS share
FROM seller_rev r JOIN sellers s ON s.id = r.seller_id;
```

Представления (VIEW)

Представление (VIEW) сохраняет запрос в базе под именем. Само по себе обычное представление не хранит данные — это сохранённый запрос; при каждом обращении к нему выполняется лежащий в основе SELECT и возвращаются актуальные данные. Представления скрывают сложность запросов, предоставляют готовые аналитические витрины и ограничивают доступ к отдельным столбцам. Материализованное представление (MATERIALIZED VIEW), напротив, физически хранит результат запроса и обновляется командой REFRESH; оно ускоряет повторное чтение дорогих агрегатов ценой неактуальности данных между обновлениями.

```
CREATE VIEW seller_revenue AS
SELECT s.id, s.name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM sellers s
JOIN products p     ON p.seller_id = s.id
JOIN order_items oi ON oi.product_id = p.id
GROUP BY s.id, s.name;
```

Техническое задание

Составьте набор многотабличных и аналитических запросов к базе данных маркетплейса и оформите наиболее полезные из них в виде представлений.

1. Напишите запрос с внутренним соединением не менее трёх таблиц (товар — категория — продавец).
2. Напишите запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице (например, товары без отзывов).
3. Постройте запрос с группировкой и агрегацией: по каждой категории — количество товаров и средняя цена.
4. Примените HAVING для фильтрации групп (например, продавцы с выручкой выше заданной).
5. Напишите по одному запросу с подзапросом каждого вида: скалярным, с IN и с EXISTS (или коррелированным).

6. Используйте CTE для разбиения сложного запроса на шаги; реализуйте рекурсивный CTE для обхода дерева категорий.
7. Создайте не менее двух представлений для типовых отчётов и продемонстрируйте обращение к ним.

Контрольные вопросы

1. Что такое соединение таблиц и по какому условию оно обычно выполняется?
2. Чем INNER JOIN отличается от LEFT JOIN? Приведите пример, где нужен именно LEFT JOIN.
3. Какие агрегатные функции вы знаете? В чём различие между COUNT(*) и COUNT(столбец)?
4. Для чего предназначена секция GROUP BY и что допустимо в списке выборки при группировке?
5. В чём различие между WHERE и HAVING и в каком порядке они выполняются?
6. Что такое подзапрос? Чем коррелированный подзапрос отличается от некоррелированного?
7. Как работает предикат EXISTS и когда его удобно применять?
8. Что такое общее табличное выражение (CTE) и для чего нужен рекурсивный CTE?
9. Что такое представление (VIEW)? Хранит ли обычное представление данные?
10. Чем материализованное представление отличается от обычного?

Лабораторная работа №4

Оконные функции и аналитика данных

Теоретическая часть

Понятие оконной функции

Оконная функция выполняет вычисление над набором строк, связанных с текущей строкой (этот набор называют окном), но, в отличие от агрегатной функции с GROUP BY, не схлопывает строки в одну. Каждая исходная строка сохраняется в результате, а рядом с ней появляется значение, вычисленное по окну. Это позволяет одновременно видеть и детальные данные, и сводные показатели — например, цену товара и его место в рейтинге внутри категории, или сумму заказа и нарастающий итог. Оконные функции вычисляются на позднем этапе обработки запроса — после WHERE, GROUP BY и HAVING, но до ORDER BY и LIMIT, поэтому в самой оконной функции уже доступны результаты группировки.

Синтаксис: конструкция OVER

Оконная функция записывается с обязательной конструкцией OVER, задающей окно. Секция PARTITION BY делит строки на разделы, внутри которых функция вычисляется независимо; она похожа на GROUP BY, но строки не объединяются, а лишь группируются для расчёта. Если PARTITION BY опущена, окном считаются все строки результата. Секция ORDER BY внутри OVER задаёт порядок строк в окне — он критичен для ранжирующих функций и накопительных итогов. Дополнительно можно задать рамку окна (frame) конструкциями ROWS или RANGE, ограничивающую набор строк относительно текущей (например, «от начала раздела до текущей строки» для накопительного итога).

Ранжирующие функции

Ранжирующие функции назначают строкам номера в пределах окна согласно порядку ORDER BY. ROW_NUMBER присваивает уникальные последовательные номера (1, 2, 3, 4) независимо от совпадения значений. RANK присваивает одинаковый ранг строкам с равными значениями сортировки, но затем пропускает следующие номера (1, 1, 3, 4). DENSE_RANK также даёт равным строкам одинаковый ранг, но не образует пропусков (1, 1, 2, 3). NTILE(n) делит строки на n примерно равных групп. Пример ранжирования товаров по цене внутри каждой категории:

```
SELECT c.name AS category, p.title, p.price,
```

```
       ROW_NUMBER() OVER w AS rn,
       RANK()       OVER w AS rnk,
       DENSE_RANK() OVER w AS dense_rnk
FROM products p JOIN categories c ON c.id = p.category_id
WINDOW w AS (PARTITION BY p.category_id ORDER BY p.price DESC);
```

Агрегатные функции в роли оконных и функции смещения

Любую агрегатную функцию можно использовать как оконную, добавив OVER. Так SUM(...) OVER (ORDER BY ...) вычисляет нарастающий итог, а SUM(...) OVER (PARTITION BY ...) — сумму по разделу, не убирая детальные строки; деление значения строки на такую сумму даёт долю в группе. Функции смещения LAG(столбец) и LEAD(столбец) возвращают значение из предыдущей или следующей строки окна и применяются для сравнения с предыдущим периодом. Функции FIRST_VALUE и LAST_VALUE возвращают первое и последнее значение в окне. Пример нарастающего итога выручки по дням:

```
SELECT o.created_at::date AS day,
       SUM(oi.quantity * oi.unit_price) AS day_revenue,
       SUM(SUM(oi.quantity * oi.unit_price))
           OVER (ORDER BY o.created_at::date) AS running_total,
       LAG(SUM(oi.quantity * oi.unit_price))
           OVER (ORDER BY o.created_at::date) AS prev_day
FROM orders o JOIN order_items oi ON oi.order_id = o.id
GROUP BY o.created_at::date
ORDER BY day;
```

Top-N в группе и сравнение с GROUP BY

Частая аналитическая задача — отобрать несколько лучших строк в каждой группе (например, три самых дорогих товара в категории). Её решают так: оконной функцией ROW_NUMBER нумеруют строки внутри раздела по нужному порядку, затем во внешнем запросе или CTE оставляют строки с номером не больше N. Оконная функция предпочтительнее обычной группировки, когда нужно сохранить детальные строки рядом со сводным показателем: GROUP BY вернул бы только агрегаты, потеряв исходные строки, тогда как оконная функция показывает и то, и другое одновременно.

Техническое задание

Реализуйте набор аналитических запросов с использованием оконных функций.

1. Постройте рейтинг товаров по цене внутри каждой категории с помощью ROW_NUMBER, RANK и DENSE_RANK; поясните, чем различаются их результаты на строках с равными значениями.

2. Для каждой категории выведите три самых дорогих товара (нумерация в окне и фильтрация через CTE).
3. Вычислите нарастающий итог выручки по дням оконной агрегатной функцией.
4. С помощью SUM(...) OVER (PARTITION BY ...) посчитайте долю выручки каждого продавца в выручке его категории.
5. Примените LAG и LEAD для сравнения выручки текущего дня с предыдущим (вычислите прирост).
6. Решите одну из задач двумя способами — оконной функцией и обычной группировкой — и поясните, почему оконная функция здесь удобнее.

Контрольные вопросы

1. Что такое оконная функция и чем она отличается от агрегатной функции с GROUP BY?
2. За что отвечает конструкция OVER?
3. Что задаёт секция PARTITION BY и чем она похожа и непохожа на GROUP BY?
4. Для чего нужна секция ORDER BY внутри OVER?
5. Чем различаются ROW_NUMBER, RANK и DENSE_RANK на строках с равными значениями?
6. Как с помощью оконной функции отобрать N лучших строк в каждой группе?
7. Как использовать агрегатную функцию в роли оконной и какой результат это даёт?
8. Что такое нарастающий итог и как его получить оконной функцией?
9. Что делают функции LAG и LEAD и для каких задач применяются?
10. В каких случаях оконная функция предпочтительнее обычной группировки?

Лабораторная работа №5

Транзакции, целостность данных и серверная логика

Теоретическая часть

Транзакции и свойства ACID

Транзакция — последовательность операций, выполняемых как единое неделимое целое: либо применяются все её изменения, либо ни одно. Транзакции обладают свойствами ACID. Атомарность (Atomicity) гарантирует принцип «всё или ничего»: при сбое на любом шаге все изменения транзакции откатываются. Согласованность (Consistency) означает, что транзакция переводит базу из одного корректного состояния в другое, не нарушая ограничений целостности. Изоляция (Isolation) обеспечивает, что параллельно выполняемые транзакции не мешают друг другу и их совместный эффект эквивалентен некоторому последовательному выполнению. Долговечность (Durability) гарантирует, что зафиксированные изменения сохранятся даже при последующем сбое.

Транзакция начинается командой BEGIN и завершается либо COMMIT, который фиксирует все изменения и делает их видимыми и постоянными, либо ROLLBACK, который отменяет изменения и возвращает базу к состоянию до начала транзакции. Команда SAVEPOINT создаёт внутри транзакции точку, к которой можно частично откатиться (ROLLBACK TO SAVEPOINT). По умолчанию каждая отдельная команда вне явного BEGIN выполняется в собственной транзакции (режим автокоммита).

Транзакции необходимы там, где несколько изменений должны произойти согласованно. В маркетплейсе оформление заказа состоит из создания заказа, его позиций и уменьшения складского остатка; если товара не хватает, все ранее сделанные изменения нужно отменить, иначе данные окажутся рассогласованы. Именно атомарность не допускает «половинчатого» заказа.

```
BEGIN;
  INSERT INTO orders (user_id) VALUES (42) RETURNING id;   -- пусть вернулось
100
  INSERT INTO order_items (order_id, product_id, quantity, unit_price)
    VALUES (100, 7, 2, 1500.00);
  UPDATE stock SET quantity = quantity - 2 WHERE product_id = 7;
COMMIT;   -- при ошибке на любом шаге выполняется ROLLBACK
```

Изоляция управляется уровнями изоляции (Read Committed по умолчанию в PostgreSQL, а также Repeatable Read и Serializable), которые определяют, какие эффекты параллельных транзакций видны текущей. PostgreSQL реализует изоляцию через механизм MVCC (многоверсионное управление конкурентным доступом): читающие транзакции видят согласованный снимок данных и не блокируют пишущие.

Функции и язык PL/pgSQL

Бизнес-логику можно размещать на стороне СУБД в виде функций. PL/pgSQL — процедурный язык PostgreSQL, расширяющий SQL управляющими конструкциями. Тело функции заключается между BEGIN и END; в разделе DECLARE объявляются локальные переменные. Поддерживаются присваивание, ветвление IF ... THEN ... ELSE, циклы (LOOP, WHILE, FOR), а также выполнение запросов с сохранением результата в переменные через SELECT ... INTO. Функция объявляет тип возвращаемого значения (RETURNS) и возвращает его командой RETURN.

Ошибочные ситуации обрабатываются командой RAISE EXCEPTION, которая прерывает выполнение и откатывает изменения текущей транзакции. Перехватить исключение можно блоком EXCEPTION внутри функции. Пример функции, атомарно оформляющей покупку с проверкой остатка: при нехватке товара возбуждается исключение, и ни одно изменение не сохраняется.

```
CREATE OR REPLACE FUNCTION place_order(p_user BIGINT, p_product BIGINT, p_qty
INT)
RETURNS BIGINT AS $$
DECLARE
    v_order_id BIGINT;
    v_price    NUMERIC(12,2);
BEGIN
    SELECT price INTO v_price FROM products WHERE id = p_product;
    IF (SELECT quantity FROM stock WHERE product_id = p_product) < p_qty THEN
        RAISE EXCEPTION 'Недостаточно товара на складе (id=%)', p_product;
    END IF;
    INSERT INTO orders (user_id) VALUES (p_user) RETURNING id INTO v_order_id;
    INSERT INTO order_items (order_id, product_id, quantity, unit_price)
        VALUES (v_order_id, p_product, p_qty, v_price);
    UPDATE stock SET quantity = quantity - p_qty WHERE product_id = p_product;
    RETURN v_order_id;
END;
$$ LANGUAGE plpgsql;
```

Триггеры

Триггер автоматически выполняет триггерную функцию в ответ на изменение данных в таблице. Событием служит INSERT, UPDATE или

DELETE. Момент срабатывания задаётся ключевыми словами BEFORE (до операции — позволяет проверить или скорректировать данные) или AFTER (после операции — удобно для аудита и пересчёта связанных данных); для представлений используется INSTEAD OF. Уровень срабатывания: FOR EACH ROW — для каждой затронутой строки, FOR EACH STATEMENT — один раз на оператор. Внутри триггерной функции доступны псевдозаписи NEW (новая версия строки) и OLD (прежняя версия).

Триггеры применяют для ведения журнала изменений (аудита), валидации данных, автоматического заполнения вычисляемых полей и поддержания денормализованных значений. Пример триггера, пересчитывающего средний рейтинг товара при добавлении отзыва:

```
CREATE OR REPLACE FUNCTION refresh_product_rating()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE products
    SET rating = (SELECT AVG(score) FROM reviews WHERE product_id =
NEW.product_id)
    WHERE id = NEW.product_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_refresh_rating
AFTER INSERT ON reviews
FOR EACH ROW EXECUTE FUNCTION refresh_product_rating();
```

Логика в базе данных или в приложении

Размещение логики в СУБД имеет достоинства: правило применяется единообразно независимо от приложения, обращающегося к базе; вычисления выполняются близко к данным, без их пересылки; целостность гарантируется на уровне хранилища. Недостатки: серверный код сложнее тестировать и отлаживать, он привязывает решение к конкретной СУБД (хуже переносимость), повышает нагрузку на сервер базы и требует отдельной дисциплины версионирования. Поэтому в БД обычно держат логику, неразрывно связанную с целостностью данных, а прикладную бизнес-логику чаще располагают в приложении.

Техническое задание

Обеспечьте согласованность данных и реализуйте серверную логику для базы данных маркетплейса.

1. Продемонстрируйте работу транзакции: выполните связанные изменения между BEGIN и COMMIT, затем повторите сценарий с намеренной ошибкой и ROLLBACK, показав, что изменения отменены.
2. Реализуйте функцию на PL/pgSQL, атомарно оформляющую заказ: она создаёт заказ и его позиции и уменьшает остаток, а при нехватке товара возбуждает исключение и не оставляет частичных изменений.
3. Реализуйте функцию, возвращающую сводный показатель (например, выручку продавца за период) с использованием переменных и запроса внутри функции.
4. Создайте триггер аудита: при изменении цены товара старое и новое значение автоматически заносятся в таблицу журнала.
5. Создайте триггер, поддерживающий денормализованное поле (например, средний рейтинг товара пересчитывается при добавлении отзыва).
6. В отчёте обоснуйте, какую логику уместно держать в базе данных, а какую — в приложении.

Контрольные вопросы

1. Что такое транзакция и какими свойствами ACID она обладает?
2. Что делают команды COMMIT и ROLLBACK? Для чего нужен SAVEPOINT?
3. Приведите пример сценария в маркетплейсе, где необходима транзакция, и объясните почему.
4. Что обеспечивает свойство атомарности и что — свойство изоляции транзакции?
5. Что такое функция в PostgreSQL и какие конструкции предоставляет язык PL/pgSQL?
6. Как в функции PL/pgSQL обработать ошибочную ситуацию и прервать выполнение?
7. Что такое триггер и в ответ на какие события он срабатывает?
8. Чем различаются триггеры BEFORE и AFTER, а также уровня строки и уровня оператора? Что такое NEW и OLD?
9. Для каких задач применяют триггеры? Приведите примеры из кейса маркетплейса.

10. Каковы достоинства и недостатки размещения бизнес-логики на стороне СУБД?

Лабораторная работа №6

Современные возможности PostgreSQL и оптимизация запросов

Теоретическая часть

Тип данных JSONB

PostgreSQL поддерживает хранение данных в формате JSON двумя типами: JSON хранит текст как есть, а JSONB — в разобранном двоичном виде. JSONB предпочтителен, поскольку обеспечивает быстрый доступ к полям, удаляет незначащие пробелы и дубли ключей и, главное, поддерживает индексирование. JSONB применяют для слабоструктурированных или изменчивых атрибутов, набор которых неудобно описывать фиксированными столбцами, — например, для характеристик товара, различающихся по категориям (у смартфона диагональ экрана, у книги число страниц). Злоупотреблять JSONB не следует: данные с устойчивой структурой и связями лучше хранить в обычных столбцах и таблицах, иначе теряются преимущества реляционной модели и контроль целостности.

К полям JSONB обращаются операторами: -> возвращает значение по ключу как JSON, ->> — как текст, #> и #>> извлекают значение по пути. Оператор @> проверяет, содержит ли документ заданный фрагмент, а оператор ? — наличие ключа. Пример добавления и запроса характеристик товара:

```
ALTER TABLE products ADD COLUMN attributes JSONB;

UPDATE products SET attributes = '{"color": "black", "weight": 180}'
WHERE id = 7;

SELECT title, attributes ->> 'color' AS color
FROM products
WHERE attributes @> '{"color": "black"}';
```

Как СУБД исполняет запрос

Получив запрос, планировщик PostgreSQL выбирает план выполнения — последовательность операций, минимизирующую оценочную стоимость. Решения опираются на статистику о данных, которую собирает команда ANALYZE (число строк, распределение значений). При выборке данных возможны разные способы доступа. Последовательное сканирование (Seq Scan) читает все строки таблицы подряд и оправдано, когда нужно вернуть

значительную их часть. Индексное сканирование (Index Scan) находит строки через индекс и эффективно при выборке небольшой доли строк. Index Only Scan возвращает данные прямо из индекса, не обращаясь к таблице. Bitmap Scan — промежуточный вариант для умеренно избирательных условий.

Индексы

Индекс — вспомогательная структура, позволяющая быстро находить строки по значению столбца, не просматривая всю таблицу. Основной и используемый по умолчанию тип — B-tree; он эффективен для сравнений на равенство и для диапазонных условий (>, <, BETWEEN) и поддерживает сортировку. Составной индекс по нескольким столбцам учитывает их порядок: он применим для условий по первому столбцу (или префиксу столбцов). Покрывающий индекс (с фразой INCLUDE) хранит дополнительные столбцы, позволяя выполнить Index Only Scan. Частичный индекс строится только по части строк, удовлетворяющих условию. Уникальный индекс дополнительно обеспечивает уникальность.

```
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_cat_price ON products(category_id, price);
CREATE INDEX idx_products_active ON products(category_id) WHERE is_active;
```

Анализ плана: EXPLAIN и EXPLAIN ANALYZE

Команда EXPLAIN показывает план выполнения запроса с оценкой стоимости каждого узла, не выполняя сам запрос. Команда EXPLAIN ANALYZE дополнительно реально выполняет запрос и показывает фактическое время и число обработанных строк; модификатор BUFFERS добавляет сведения об обращениях к страницам данных. План читается как дерево узлов; по типу узла видно, используется ли индекс: появление Index Scan или Index Only Scan вместо Seq Scan по нужному столбцу означает, что индекс задействован. Сопоставляя планы до и после добавления индекса, оценивают его эффект.

```
EXPLAIN ANALYZE
SELECT title, price FROM products
WHERE category_id = 5 AND price < 3000;
```

Цена индексов и GIN

Индексы ускоряют чтение, но имеют цену: они занимают место на диске и замедляют операции вставки, обновления и удаления, поскольку индекс приходится поддерживать в актуальном состоянии. Поэтому индексы создают не на все столбцы подряд, а осознанно — под конкретные

частые запросы. Снижают эффективность индекса и антипаттерны: применение функции к индексируемому столбцу в условии или неявное приведение типов часто лишают планировщик возможности использовать индекс (помогает индекс по выражению или приведение литерала к типу столбца).

Для индексирования JSONB, массивов и полнотекстового поиска применяется индекс типа GIN (Generalized Inverted Index, обобщённый инвертированный индекс). Он эффективно поддерживает проверку вхождения и поиск по элементам составных значений (оператор @> для JSONB), для которых обычный B-tree не подходит. После создания GIN-индекса запросы с оператором @> могут использовать его, что видно по плану выполнения.

```
CREATE INDEX idx_products_attrs ON products USING GIN (attributes);

EXPLAIN ANALYZE
SELECT title FROM products WHERE attributes @> '{"color": "black"}';
```

Техническое задание

Освойте современные возможности PostgreSQL и оптимизируйте запросы к базе данных маркетплейса.

1. Добавьте к товарам столбец типа JSONB для хранения характеристик и заполните его для разных категорий товаров.
2. Напишите запросы, извлекающие отдельные поля из JSONB (операторы -> и ->>) и фильтрующие товары по вхождению фрагмента (оператор @>).
3. Выберите два-три «тяжёлых» запроса из предыдущих работ и снимите их планы командой EXPLAIN ANALYZE; зафиксируйте тип сканирования и время.
4. Добавьте подходящие B-tree-индексы и повторно снимите планы; сравните время и тип сканирования до и после, объясните изменение плана.
5. Создайте GIN-индекс по столбцу JSONB и покажите на плане, что запрос с оператором @> стал использовать индекс.
6. Продемонстрируйте обратную сторону индексов: сравните время массовой вставки данных в таблицу с индексами и без них.
7. В отчёте обоснуйте, какие индексы и почему вы добавили, опираясь на планы выполнения.

Контрольные вопросы

1. Что такое тип JSONB, чем он отличается от JSON и для каких данных подходит?
2. Какими операторами извлекают поля из JSONB и проверяют вхождение фрагмента?
3. Что такое индекс и как он ускоряет выборку данных?
4. Чем последовательное сканирование (Seq Scan) отличается от индексного (Index Scan)?
5. Какой тип индекса используется по умолчанию и для каких условий он эффективен?
6. Что показывает команда EXPLAIN и чем от неё отличается EXPLAIN ANALYZE?
7. Как по плану выполнения определить, используется ли индекс?
8. Почему индексы не следует создавать на все столбцы подряд? Какова их цена?
9. Что такое GIN-индекс и в каких случаях он применяется?
10. Как индексы влияют на операции вставки и обновления данных?

Лабораторная работа №7

Итоговый проект

Общее описание

Итоговая работа представляет собой комплексный проект базы данных маркетплейса, объединяющий все технологии, изученные в течение курса. Проект развивает базу данных, созданную и дополнявшуюся в предыдущих работах, и доводит её до состояния законченного решения с документацией. Цель работы — показать умение самостоятельно спроектировать, реализовать, наполнить и оптимизировать базу данных, а также обосновать принятые решения при защите.

Состав работы

В рамках итогового проекта студент должен:

1. представить завершённую схему базы данных в третьей нормальной форме со всеми ограничениями целостности, оформленную единым воспроизводимым DDL-скриптом;
2. наполнить базу согласованными данными в объёме, достаточном для содержательной аналитики;
3. реализовать набор SQL-запросов разной сложности: многотабличные выборки, агрегацию, подзапросы, CTE и оконные функции;
4. создать аналитические представления для типовых отчётов площадки;
5. реализовать механизмы обеспечения целостности и серверную логику: транзакционные функции на PL/pgSQL и триггеры;
6. применить современные возможности PostgreSQL: хранение характеристик товаров в JSONB и соответствующие запросы;
7. оптимизировать ключевые запросы: подобрать индексы на основе анализа планов выполнения и подтвердить эффект командой EXPLAIN ANALYZE;
8. подготовить документацию (описание схемы, словарь данных, перечень запросов и обоснование проектных решений) и защитить проект.

Критерии оценивания

При оценке проекта учитываются: корректность и полнота схемы данных; соблюдение целостности; правильность и эффективность запросов; обоснованность выбора индексов по планам выполнения; качество

серверной логики и её устойчивость к ошибочным сценариям; полнота документации; умение при защите объяснить принятые решения, а не только продемонстрировать результат.